AI-Powered Smart Vehicle Rental Management System

A Mini Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Kompally Gyaneshwar (Team Lead) – 2420030215

Kota Hari Prasanna Teja - 2420030244

Pusapati Dheeraj -2420090123

Under the Esteemed Guidance of

Dr. Mary Navyatha Govindu



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**AI-Powered Smart Vehicle Rental Management System**

"is a record of Bonafide work of **kompally Gyaneshwar – 2420030215, kota Hari Prasanna Teja– 2420030244, Pusapati Dheeraj – 2420090123** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Kompally Gyaneshwar – 2420030215

Kota Hari Prasanna Teja – 2420030244

Pusapati Dheeraj Varma– 2420090123

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Or

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

CERTIFICATE

This is certify that the mini project based report entitled "**AI-Powered Smart Vehicle Rental Management System"** is a bonafide work done and submitted by **kompally Gyaneshwar-2420030215 , Kota Hari Prasanna Teja , Pusapati Dheeraj- 2420090123** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Course Co-ordinator **Signature of HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, **Dr. Navyatha Govindu**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Sir (CSE)** Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Kompally Gyaneshwar – 2420030215

Kota Hari Prasanna Teja – 2420030244

Pusapati Dheeraj Varma– 2420090123

.

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	12
6.	Implementation	13
7.	Results & Discussion	23
8.	Conclusion & Future Scope	26
9.	References	27

1. ABSTRACT

Traditional vehicle rental systems often face challenges in managing multiple rental shops, vehicle availability, bookings, payments, and customer services. Monolithic architectures tightly connect these functions, making the system difficult to maintain, update, and scale. Additionally, customers may struggle to find suitable vehicles, check availability, and receive personalized recommendations. To overcome these limitations, this project proposes an AI Smart Vehicle Rental System, a multi-city vehicle rental platform developed using a microservices architecture.

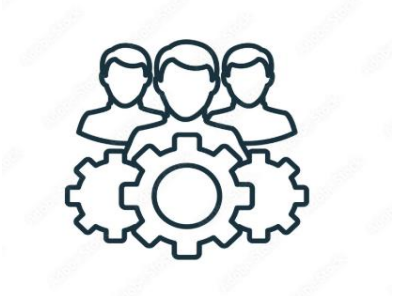
The proposed system consists of independent services, including User, Shop, Vehicle, Booking, Payment, Document, Notification, and AI Recommendation services. Each service maintains its own database and communicates through REST APIs using Spring Cloud API Gateway and Netflix Eureka for request routing and service discovery. The AI component is developed using Python FastAPI and applies machine learning techniques to recommend suitable vehicles based on customer preferences, requirements, and rental history. An AI-powered chatbot assists customers with rental-related queries. The platform also supports shop approval, document verification, multi-branch pickup and return, booking management, offline payment tracking, automated overdue notifications, late-fee management, and vehicle condition inspections.

The system aims to improve scalability, flexibility, security, and customer experience through independent services and AI-based assistance. Overall, this project provides a smart, reliable, and efficient solution for modern multi-city vehicle rental operations.

Team Name:

404 Competition Not Found

Team Logo (if any):



Team Members:

Kompally Gyaneshwar-2420030215

Kota Hari Prasanna Teja-2420030244

Pusapati Dheeraj -2420090123

2. INTRODUCTION

In today's digital world, vehicle rental services are becoming increasingly important for people who need temporary transportation for travel, business, and personal purposes. Traditional vehicle rental systems often depend on manual processes or basic online platforms to manage vehicles, customer details, bookings, and payments. As rental businesses expand across multiple cities and locations, managing these operations becomes more complex. Customers may also face difficulties in finding suitable vehicles, checking availability, comparing rental options, and receiving quick assistance.

To overcome these challenges, the AI Smart Vehicle Rental System is developed as a modern, intelligent, and user-friendly vehicle rental platform. The main objective of this project is to connect customers with authorized rental shops across multiple cities and simplify the complete vehicle rental process. Customers can search for vehicles based on their preferred city, select rental shops, check vehicle availability, book vehicles, and upload the required documents for verification.

The system follows a microservices architecture, where different functionalities are divided into independent services such as User, Shop, Vehicle, Booking, Payment, Document, Notification, and AI Recommendation services. Each service operates independently and maintains its own database. Spring Boot is used for backend development, while Spring Cloud API Gateway and Netflix Eureka support request routing and service discovery. React.js and Vite are used to develop the frontend interface.

Artificial Intelligence is an important part of this project. A Python-based AI recommendation engine suggests suitable vehicles based on customer preferences and requirements. An AI-powered chatbot helps users with rental-related questions and provides quick assistance. The

platform also includes automated overdue notifications, late-fee tracking, double-booking prevention, document verification, and vehicle condition inspections.

Overall, the project aims to improve customer convenience, simplify rental shop operations, and provide a scalable and reliable vehicle rental solution. The following sections explain the system architecture, technologies used, major modules, implementation, and expected outcomes.

3. LITERATURE SURVEY

1.Zoomcar

Zoomcar is an online self-drive car rental platform that allows customers to rent vehicles for different durations. Users can search for cars based on their location, select pickup and return dates, and choose vehicles according to their requirements. The platform provides multiple vehicle options and online booking facilities. It simplifies the rental process by allowing customers to complete reservations digitally. However, it does not mainly focus on AI-based personalized vehicle recommendations.

2. Revv

Revv is a car rental service which provides cars on hourly, daily, weekly and monthly basis. Customers can choose the cars they require and book them online. The service also provides customers with vehicle delivery and collection. The service provides customers with an easy way to rent cars. The main feature of this service does not include intelligent recommendations of cars using AI, and automated fleet management.

3. Turo

Turo is a peer-to-peer car rental service, where customers can view available cars from vehicle owners. Vehicle owners can make their cars available to rent by adding details, cost and availability. Customers can view cars available in their location. Online booking can be done and flexible rental options provided. The main feature of this service does not include multiple car rental shops.

4. Getaround

Getaround is a car sharing service. Customers can view nearby cars. They can rent cars for short or long term periods using the app. Customers can have online bookings and gain digital access to the vehicle. This improves the convenience for vehicle owners and customers. The service does not provide the full microservice based approach to car rentals.

5. IndusGo

IndusGo is a self drive car rental service within India. It provides various types of vehicles. Customers can choose where to collect and drop off vehicles and can book vehicles for varying rental periods. The service also provides delivery and document verification. This helps customers gain access to rental cars in various locations. The recommendations can be improved using intelligent vehicles and automated fleet.

6. MyChoize

MyChoize is an online self drive car rental service in India. It provides various categories of vehicles. Customers can book cars via the website. Customers can choose cars to suit their needs. Online bookings can be made and driving licence verification. Intelligent recommendations and automating overdue vehicles is not the main feature of this service.

7. SIXT

SIXT is an international car rental company. They provide cars, SUVs and vans. Customers can search for vehicles in their desired rental locations. Online bookings can be made and flexible rental options provided. Services such as vehicle subscriptions and airport transfers can be provided. The car rental approach differs from the intelligent multi shop vehicle management approach.

8. Hertz

Hertz is a popular international car rental company. They have many locations for customers to gain access to vehicle rentals. Customers can choose vehicles to suit their needs and choose collection and drop off dates. Online bookings can be made and changes to bookings. Varying categories of vehicles are available for personal and business travel. Personalised recommendations using AI and multi shop integration can be improved.

4. Problem Statement and Objectives

Problem Statement: Traditional vehicle rental systems lack scalable management, real-time availability, and personalized recommendations, making rental operations inefficient for customers and shop owners.

Problem Description: Managing vehicle availability, multiple rental shops, bookings, document verification, and overdue returns manually is challenging. Customers also lack personalized vehicle recommendations and a smooth rental experience.

Project Purpose

The main purpose of the AI Smart Vehicle Rental System is to develop a centralized vehicle rental platform that connects customers with authorized rental shops across multiple cities. Traditional rental systems often face difficulties in managing vehicle availability, bookings, customer documents, payments, and overdue returns. To overcome these challenges, the proposed system uses a microservices architecture in which each major function operates as an independent service with its own database.

The system also integrates an AI-powered recommendation engine that suggests suitable vehicles based on customer preferences, such as vehicle type, budget, fuel preference, and seating capacity. An AI chatbot helps customers by understanding their rental requirements and providing relevant assistance. Automated booking management, document verification, overdue monitoring, notifications, and vehicle inspections help improve the efficiency of rental operations.

Objectives:

1. Multi-City Vehicle Rental Platform

The system aims to connect customers with authorized rental shops across multiple cities. Customers can select their preferred city, view approved rental shops, browse available vehicles, and make reservations based on their requirements.

2. Independent Microservices Architecture

The project uses microservices architecture to divide the application into independent services, such as User, Shop, Vehicle, Booking, Payment, Document, and Notification Services. Each service has its own database and responsibilities, making the application easier to maintain, update, and scale independently.

3. Secure Authentication and Role-Based Access

The system provides secure registration and login facilities using Spring Security and JWT authentication. It supports three roles: CUSTOMER, SHOP_OWNER, and SUPER_ADMIN. Each role receives access only to the features and operations related to its responsibilities.

4. Vehicle Search and Booking Management

Customers can search for vehicles using filters such as city, vehicle type, fuel type, budget, and seating capacity. They can check vehicle availability, select rental dates, and submit booking requests. The system manages the complete booking process, including confirmation, pickup, return, and cancellation.

5. Double-Booking Prevention

The system checks existing reservations before confirming a new booking. It uses transaction isolation and booking time-overlap validation to prevent two customers from reserving the same vehicle for overlapping rental periods. This helps maintain accurate vehicle availability.

6. Physical Document Verification

Customers are required to upload their driving licence and other necessary documents through the platform. During the vehicle rental process, the shop owner physically checks the original documents provided by the customer and verifies whether the details match the uploaded documents. After checking, the shop owner approves or rejects the documents through the system. This helps ensure proper customer verification before handing over the vehicle.

7. Offline Payment and Deposit Management

The system allows shop owners to record payments received at the rental counter through Cash, UPI, or Card. It manages rental charges, refundable security deposits, overdue penalties, and damage deductions. It also supports digital receipt and invoice generation.

8. Automated Overdue Monitoring and Notifications

The system uses Spring Scheduler to monitor active bookings and identify vehicles that are not returned before the scheduled time. When a booking becomes overdue, the system updates its status, calculates late charges, and sends notifications to customers. Duplicate notifications are controlled to avoid unnecessary alerts.

9. Vehicle Pickup and Return Inspection

The system allows shop owners to record vehicle condition details during pickup and return. Information such as odometer readings, fuel percentage, scratches, and damage details can be recorded. Any new damage charges can be considered during security deposit settlement.

10. AI-Based Vehicle Recommendations and Chatbot

The project integrates a Python FastAPI-based AI service that provides personalized vehicle recommendations using a weighted feature similarity algorithm. It considers vehicle type, budget, fuel preference, and seating capacity to rank suitable vehicles. The chatbot also extracts customer requirements from natural language and helps users find relevant rental options.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i5 or equivalent processor.
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: Minimum 20 GB of free disk space.
- System: Desktop or laptop with a stable internet connection.

Software Requirements:

- Operating System: Windows 10/11 or Linux.
- Frontend: React.js, Vite, Tailwind CSS, and Node.js.
- Backend: Java 17/21, Spring Boot 3.2.5, Spring Cloud, and Maven.
- Database: MySQL 8.0.
- AI Service: Python 3.11+, FastAPI, and Uvicorn.
- Tools: Visual Studio Code, IntelliJ IDEA, Postman, and Git.

6.IMPLEMENTATION

1. Frontend (HTML, CSS, JavaScript):

- **index.html:**

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8" />

<link rel="icon" type="image/svg+xml" href="/vite.svg" />

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<title>AI Smart Vehicle Rental System</title>

<!-- Google Fonts -->

<link rel="preconnect" href="https://fonts.googleapis.com">

<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700
;800&display=swap" rel="stylesheet">

</head>

<body class="bg-slate-50 text-slate-800 antialiased font-sans">

<div id="root"></div>

<script type="module" src="/src/main.jsx"></script>

</body>

</html>
```

. index.css:

```
@tailwind base;
@tailwind components;
```

```
@tailwind utilities;

@layer base {
body {
@apply bg-slate-50 text-slate-800 font-sans;
}
}

/* Custom scrollbars */
::-webkit-scrollbar {
width: 6px;
height: 6px;
}
::-webkit-scrollbar-track {
background: #f1f5f9;
}
::-webkit-scrollbar-thumb {
background: #cbd5e1;
border-radius: 9999px;
}
::-webkit-scrollbar-thumb:hover {
background: #94a3b8;
}
```

App.jsx:

```
import React from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';

import Navbar from './components/Navbar';
import Footer from './components/Footer';
import AIChatWidget from './components/AIChatWidget';
import ProtectedRoute from './components/ProtectedRoute';

// Auth Pages
import Login from './pages/auth/Login';
import RegisterCustomer from './pages/auth/RegisterCustomer';
import RegisterShopOwner from './pages/auth/RegisterShopOwner';

// Customer Pages
import CustomerHome from './pages/customer/CustomerHome';
import CityShops from './pages/customer/CityShops';
import ShopVehicles from './pages/customer/ShopVehicles';
import VehicleDetails from './pages/customer/VehicleDetails';
import MyBookings from './pages/customer/MyBookings';
import BookingDetail from './pages/customer/BookingDetail';
```

```
import CustomerProfile from './pages/customer/CustomerProfile';
import CustomerComplaints from './pages/customer/CustomerComplaints';

// Shop Owner Pages
import ShopDashboard from './pages/shop/ShopDashboard';
import ShopVehiclesManager from './pages/shop/ShopVehiclesManager';
import ShopBookingsManager from './pages/shop/ShopBookingsManager';
import ShopInspectionPickupReturn from './pages/shop/ShopInspectionPickupReturn';
import ShopPaymentsManager from './pages/shop/ShopPaymentsManager';
import ShopProfileSettings from './pages/shop/ShopProfileSettings';

// Super Admin Pages
import AdminDashboard from './pages/admin/AdminDashboard';
import AdminShopsManager from './pages/admin/AdminShopsManager';
import AdminUsersManager from './pages/admin/AdminUsersManager';
import AdminBookingsMonitor from './pages/admin/AdminBookingsMonitor';
import AdminComplaintsManager from './pages/admin/AdminComplaintsManager';

export default function App() {
return (
<div className="min-h-screen flex flex-col bg-slate-50 text-slate-800">

<Navbar />

<main className="flex-grow">
<Routes>

{/* Public Routes */}
<Route path="/" element={<CustomerHome />} />

<Route path="/login" element={<Login />} />

<Route
path="/register/customer"
element={<RegisterCustomer />}
/>

<Route
path="/register/shop-owner"
element={<RegisterShopOwner />}
/>

{/* Customer Browsing Routes */}
<Route path="/shops/:city" element={<CityShops />} />

<Route
path="/shop/:id/vehicles"
```

```
element={<ShopVehicles />}
/>

{/* Individual Vehicle Details */}
<Route
path="/vehicle/:id"
element={<VehicleDetails />}
/>

{/* Protected Customer Routes */}
<Route
path="/customer/my-bookings"
element={
<ProtectedRoute allowedRoles={['CUSTOMER']}>
<MyBookings />
</ProtectedRoute>
}
/>

<Route
path="/customer/bookings/:id"
element={
<ProtectedRoute allowedRoles={['CUSTOMER']}>
<BookingDetail />
</ProtectedRoute>
}
/>

<Route
path="/customer/profile"
element={
<ProtectedRoute allowedRoles={['CUSTOMER']}>
<CustomerProfile />
</ProtectedRoute>
}
/>

<Route
path="/customer/complaints"
element={
<ProtectedRoute allowedRoles={['CUSTOMER']}>
<CustomerComplaints />
</ProtectedRoute>
}
/>

{/* Protected Shop Owner Routes */}
```

```
<Route
path="/shop/dashboard"
element={
<ProtectedRoute allowedRoles={['SHOP_OWNER']}>
<ShopDashboard />
</ProtectedRoute>
}
/>

<Route
path="/shop/vehicles"
element={
<ProtectedRoute allowedRoles={['SHOP_OWNER']}>
<ShopVehiclesManager />
</ProtectedRoute>
}
/>

<Route
path="/shop/bookings"
element={
<ProtectedRoute allowedRoles={['SHOP_OWNER']}>
<ShopBookingsManager />
</ProtectedRoute>
}
/>

<Route
path="/shop/inspections"
element={
<ProtectedRoute allowedRoles={['SHOP_OWNER']}>
<ShopInspectionPickupReturn />
</ProtectedRoute>
}
/>

<Route
path="/shop/payments"
element={
<ProtectedRoute allowedRoles={['SHOP_OWNER']}>
<ShopPaymentsManager />
</ProtectedRoute>
}
/>

<Route
path="/shop/settings"
```

```
element={
<ProtectedRoute allowedRoles={['SHOP_OWNER']}>
<ShopProfileSettings />
</ProtectedRoute>
}
/>

{/* Protected Super Admin Routes */}
<Route
path="/admin/dashboard"
element={
<ProtectedRoute allowedRoles={['SUPER_ADMIN']}>
<AdminDashboard />
</ProtectedRoute>
}
/>

<Route
path="/admin/shops"
element={
<ProtectedRoute allowedRoles={['SUPER_ADMIN']}>
<AdminShopsManager />
</ProtectedRoute>
}
/>

<Route
path="/admin/users"
element={
<ProtectedRoute allowedRoles={['SUPER_ADMIN']}>
<AdminUsersManager />
</ProtectedRoute>
}
/>

<Route
path="/admin/bookings"
element={
<ProtectedRoute allowedRoles={['SUPER_ADMIN']}>
<AdminBookingsMonitor />
</ProtectedRoute>
}
/>

<Route
path="/admin/complaints"
element={
```

```
<ProtectedRoute allowedRoles={['SUPER_ADMIN']}>
<AdminComplaintsManager />
</ProtectedRoute>
}
/>

{/* Fallback Route */}
<Route path="*" element={<Navigate to="/" replace />} />

</Routes>
</main>

<AIChatWidget />

<Footer />

</div>
);
}
```

main.jsx:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './index.css';
import { AuthProvider } from './context/AuthContext';
import { NotificationProvider } from './context/NotificationContext';

ReactDOM.createRoot(document.getElementById('root')).render(
<React.StrictMode>
<BrowserRouter>
<AuthProvider>
<NotificationProvider>
<App />
</NotificationProvider>
</AuthProvider>
</BrowserRouter>
</React.StrictMode>
);
```

2. Backend (JAVA):

The backend is developed using Java and Spring Boot with a microservices architecture. Each service handles a specific functionality and maintains its own database.

A. Java Code Files

1. User Service – Port 8081:

- UserController.java – Handles customer and shop owner registration, login, and user-related API requests.
- UserService.java – Implements user registration, authentication, and account management.
- UserRepository.java – Performs database operations related to users.
- User.java – Defines user details and entity information.

2. Shop Service – Port 8082:

- ShopController.java – Handles shop registration and shop management requests.
- ShopService.java – Manages shop profiles, approval, rejection, and suspension.
- ShopRepository.java – Performs database operations related to rental shops.
- Shop.java – Defines rental shop information.

3. Vehicle Service – Port 8083:

- VehicleController.java – Handles vehicle-related API requests.
- VehicleService.java – Manages vehicle registration, updates, availability, and filtering.
- VehicleRepository.java – Performs vehicle database operations.
- Vehicle.java – Defines vehicle details and information.

4. Booking Service – Port 8084

- BookingController.java – Handles booking creation and booking-related requests.
- BookingService.java – Manages booking status, rental periods, and double-booking prevention.
- BookingRepository.java – Performs booking database operations.
- OverdueMonitoringScheduler.java – Automatically monitors overdue bookings and calculates late penalties.

5. Payment Service – Port 8085

- PaymentController.java – Handles payment-related API requests.
- PaymentService.java – Manages offline payments, security deposits, refunds, and settlements.
- PaymentRepository.java – Performs payment database operations.
- Payment.java – Defines payment information.

6. Notification Service – Port 8086

- NotificationController.java – Handles notification-related requests.

- NotificationService.java – Manages booking confirmations and overdue alerts.
- NotificationRepository.java – Performs notification database operations.

7. Document Service – Port 8087

- DocumentController.java – Handles document uploading, retrieval, and related requests.
- DocumentService.java – Manages document storage and verification status.
- DocumentRepository.java – Performs document database operations.
- Document.java – Defines uploaded document information.

B. API Integration:

API integration is used to establish communication between the React frontend, Spring Boot microservices, and Python FastAPI AI service. The Spring Cloud API Gateway acts as the central entry point for frontend requests and routes them to the appropriate microservices. Netflix Eureka provides service discovery, allowing the microservices to register and locate one another.

Integration Code Files:

File name	Purpose
`ApiGatewayApplication.java`	Starts the Spring Cloud API Gateway.
`EurekaServerApplication.java`	Starts the Eureka Server for service registration and discovery.
`application.properties`	Stores database connections, service ports, and application configurations.
`application.yml`	Defines API Gateway routes and service onfigurations, if used.
`SecurityConfig.java`	Configures Spring Security and access permissions, where implemented.
`JwtAuthenticationFilter.java`	Handles JWT validation, if implemented in that service.
`api.js`	Uses Axios to connect the React frontend with backend REST APIs.
`AIChatWidget.jsx`	Connects the frontend chatbot with the Python FastAPI AI service.

C. AI Service Integration (Python FastAPI)

The AI service is developed separately using Python and FastAPI. It provides vehicle recommendations and chatbot functionality.

Python file	Purpose
`main.py`	Defines FastAPI application and REST API endpoints.

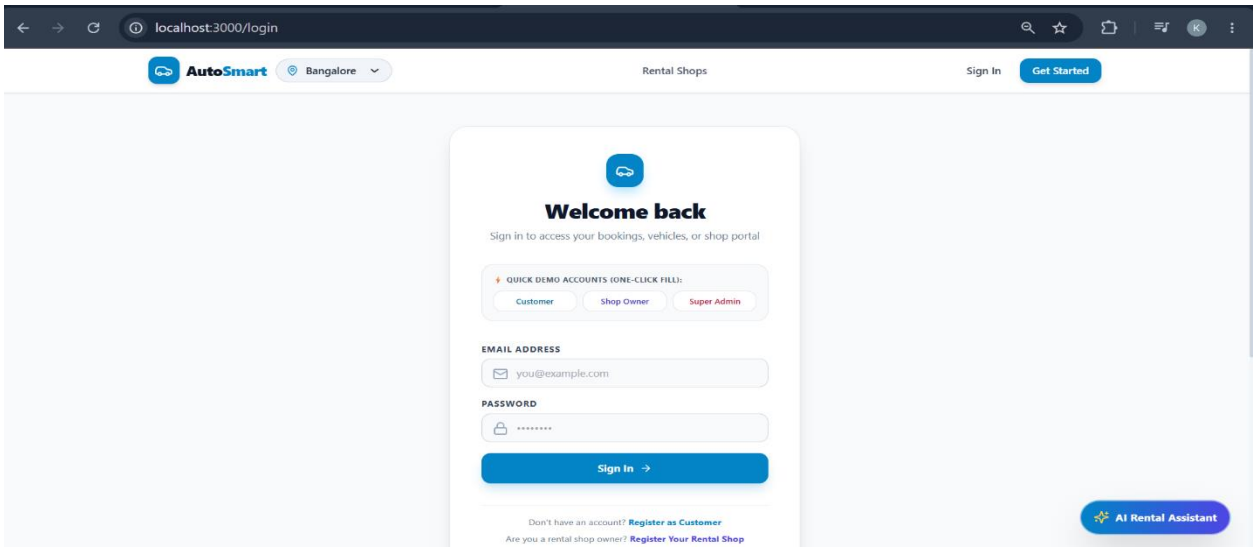
`recommender.py`	Implements the weighted multi-attribute vehicle recommendation algorithm.
`chatbot.py`	Extracts customer requirements from natural language and provides chatbot assistance.
`requirements.txt`	Contains the required Python libraries and dependencies.

D. Database Integration

The project uses MySQL 8.0 with seven separate database schemas. Each microservice manages its own database to maintain service independence.

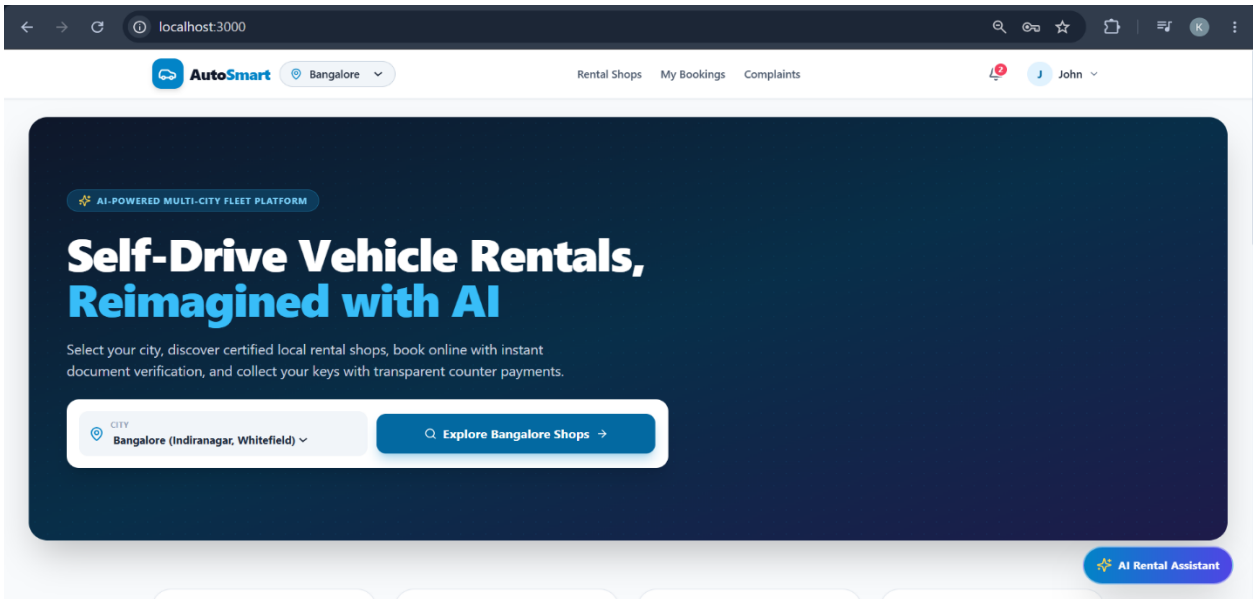
Database	Connected service
`rental_users`	User Service
`rental_shops`	Shop Service
`rental_vehicles`	Vehicle Service
`rental_bookings`	Booking Service
`rental_payments`	Payment Service
`rental_notifications`	Notification Service
`rental_documents`	Document Service

7.RESULTS & DISCUSSION



1.User Registration and Login Page

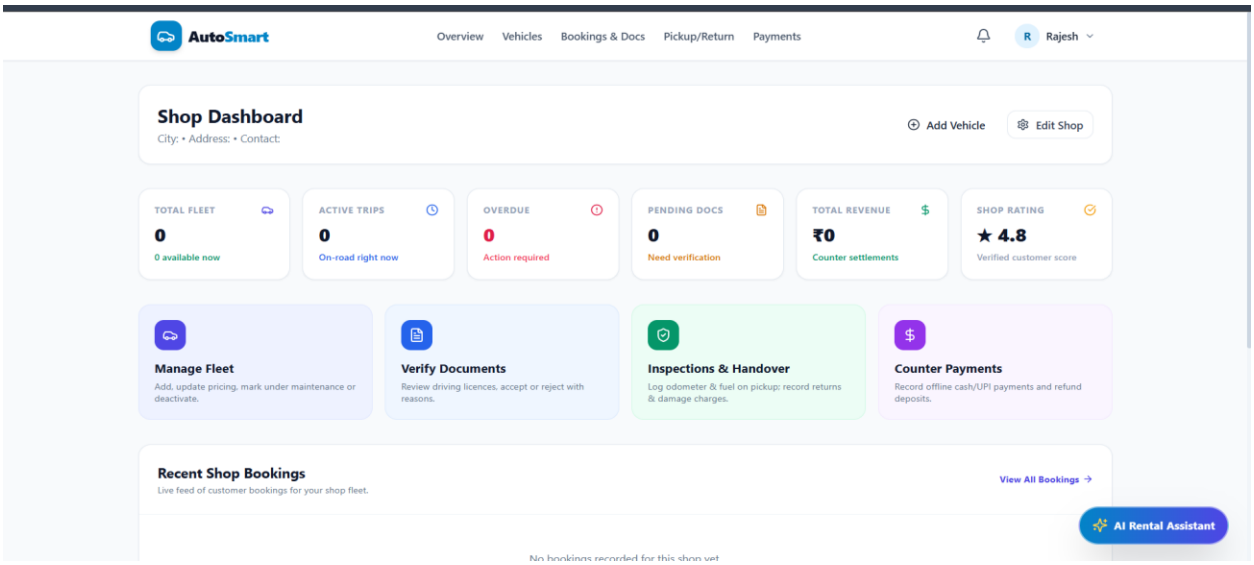
- Allows customers and shop owners to create accounts and securely log in.
- Provides role-based access to the system.



2.Home Page

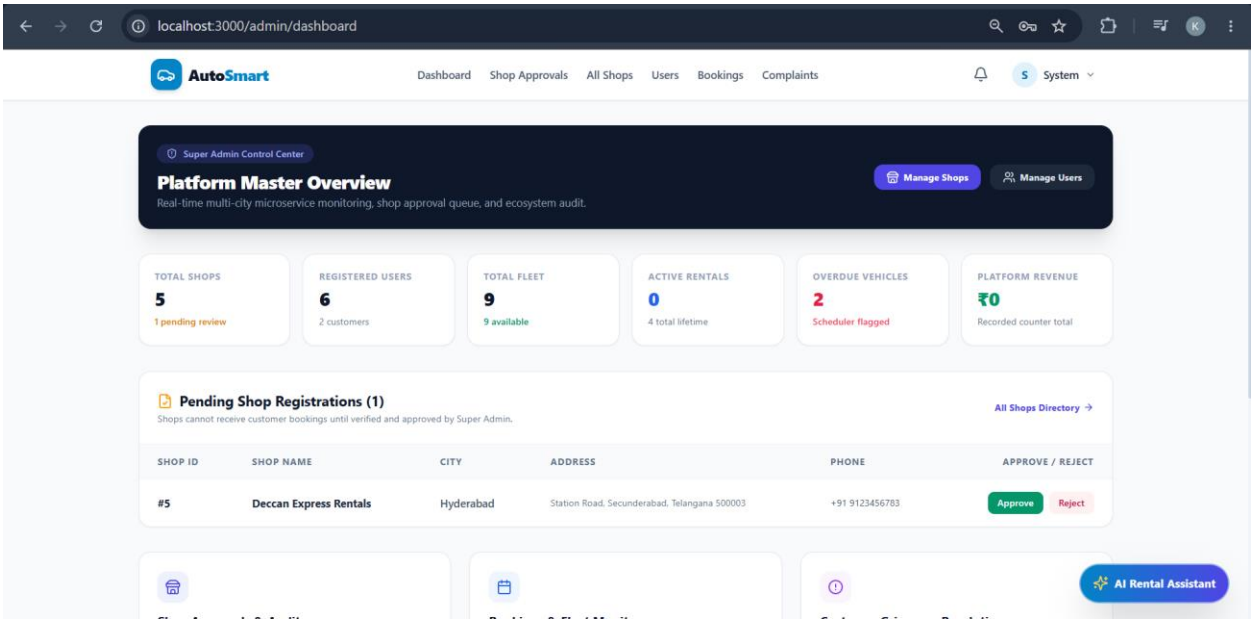
- Displays the main interface of the AI Smart Vehicle Rental System.

- Allows customers to explore rental services and start booking vehicles.



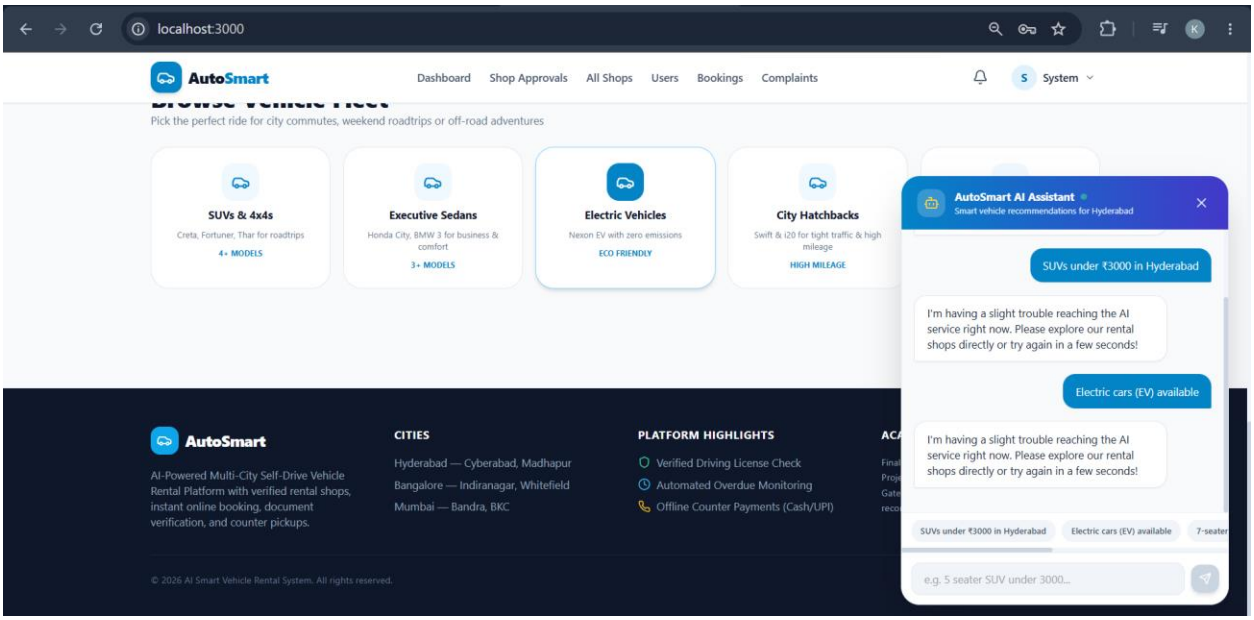
3.Shop Owner Dashboard

- Provides shop owners with an overview of vehicles, bookings, and customers.
- Helps manage daily rental operations efficiently.



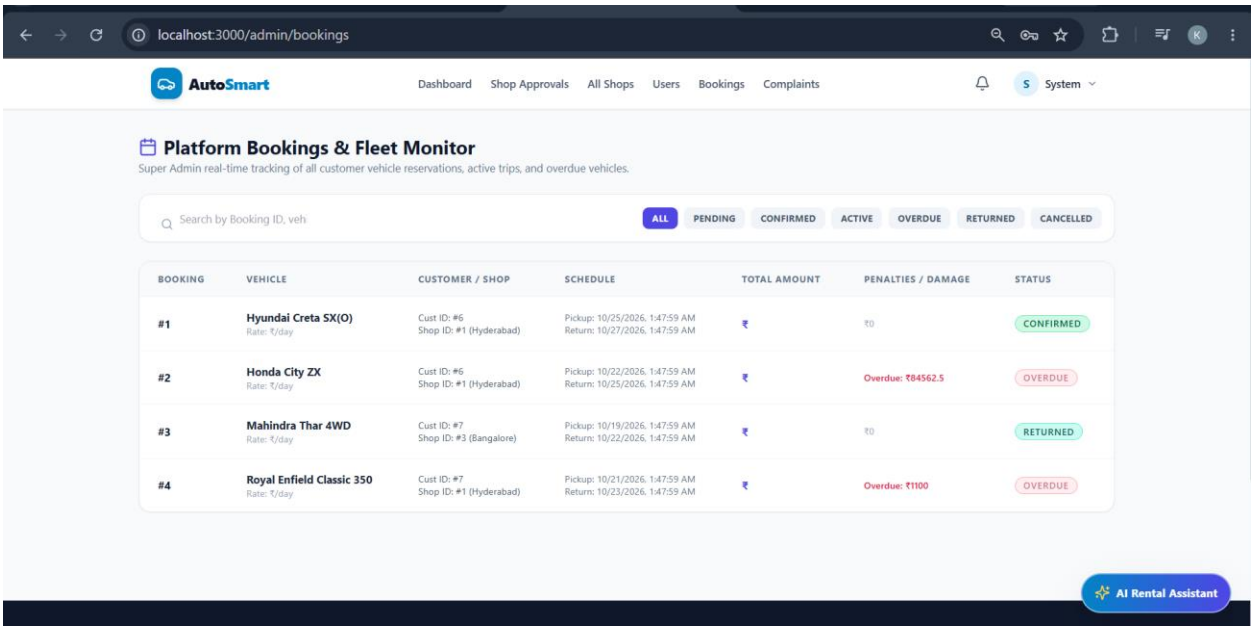
4.Super Admin Dashboard

- Displays system-wide information about users, shops, and bookings.
- Allows administrators to approve shops and manage platform activities.



5.AI Chatbot Page

- Provides an AI-powered assistant to answer customer queries.
- Helps customers find suitable vehicles and understand rental services.



6.Booking Management Page

- Displays booking details and their current status.
- Allows customers to track pending, confirmed, active, and completed rentals.

Discussion

The AI Smart Vehicle Rental System provides a centralized platform to simplify and improve the traditional vehicle rental process. The project combines modern web technologies, microservices architecture, and AI-based features to provide an efficient rental management solution. Customers can search for available vehicles, compare rental options, upload required documents, and manage their bookings through a single platform. Shop owners can manage vehicles, verify customer documents, monitor bookings, and handle vehicle inspections, while super administrators can manage users and approve rental shops. The integration of AI-powered chatbot assistance and vehicle recommendations improves customer convenience and decision-making. Additionally, automated notifications and double-booking prevention help reduce manual work and improve operational reliability. Overall, the project demonstrates how AI and modern software technologies can be combined to create a scalable, user-friendly, and efficient vehicle rental management system.

6. CONCLUSION & FUTURE SCOPE

Conclusion:

The AI Smart Vehicle Rental System successfully addresses the challenges of traditional vehicle rental processes by providing a centralized, user-friendly, and intelligent platform. The project achieves its main objectives by simplifying vehicle discovery, booking, document verification, and rental management through a microservices-based architecture. The integration of AI-powered assistance, vehicle recommendations, automated notifications, and damage inspection improves the overall rental experience. The system also provides dedicated functionalities for customers, shop owners, and super administrators. Overall, this project demonstrates how modern web technologies and AI can make vehicle rental services more efficient, reliable, and accessible.

Future Scope:

The AI Smart Vehicle Rental System can be further enhanced by introducing advanced technologies and additional functionalities to improve its performance, security, and user experience. Future developments may include a dedicated mobile application for Android and iOS, improved AI chatbot capabilities, and more personalized vehicle recommendations based on customer preferences. The system can also integrate secure online payment gateways, GPS-based real-time vehicle tracking, and AI-powered vehicle damage detection using computer vision. Furthermore, predictive maintenance can help rental shop owners identify vehicle issues in

advance, while advanced machine learning models can support demand forecasting and fleet optimization. Blockchain technology could also be explored to maintain secure and transparent rental records and transactions. In the long term, the platform can expand to multiple regions and international markets, evolving into a complete intelligent vehicle rental ecosystem that provides efficient, secure, and convenient services for customers and rental businesses.

7. REFERENCES

1. Zoomcar- https://www.zoomcar.com/

2. Revv- https://www.revv.co.in/

3. Turo- https://turo.com/

4.Getaround- https://getaround.com

5.IndusGo- https://www.indusgo.in

6.My Choize- https://www.mychoize.com

7.SIXT – https://www.sixt.com

8.Hertz – https://link.hertz.com